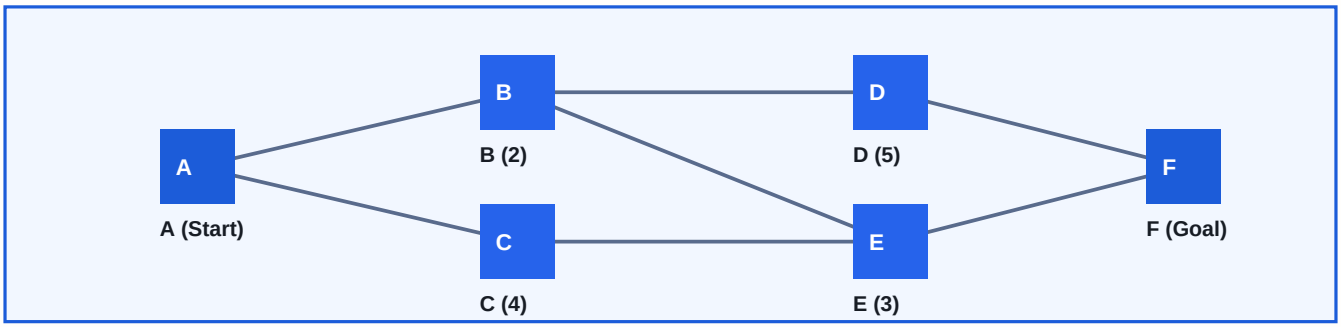
CS NOTES & ALGORITHMS

Visual Graph Study Notes

BFS, DFS, shortest path algorithms, topological ordering, cycle detection, and visual traversal traces across 8 comprehensive pages.

Algorithm Visual Lab | 8-PAGE COMPLETE DEMO GUIDE

1. Graphs, Vertices & Adjacency Lists



Formal Graph Definition

A graph G = (V, E) captures discrete entities as vertices V and pairwise connections as edges E. Edges may be undirected (symmetric roads between cities) or directed (one-way prerequisite links between courses), and weighted with numeric costs such as latency or distance.

Choosing Storage Representation

Most real-world networks - social graphs, citation webs, dependency trees - are sparse: |E| << |V|^2. Adjacency lists store only actual outgoing neighbors per vertex using O(V + E) memory rather than O(V^2) empty cells in an adjacency matrix.

REPRESENTATION TRADE-OFFS

1. Adjacency List: O(V + E) space, fast neighbor iteration, ideal for BFS/DFS
2. Adjacency Matrix: O(V^2) space, O(1) edge lookup u -> v, suits dense graphs
3. Edge List: Compact tuple stream [u, v, w], ideal for Kruskal MST sorting
4. Degree Invariant: Sum of undirected degrees always equals 2 * |E|

2. Breadth-First Search & Shortest Hops

Wavefront Expansion by Layer

Breadth-First Search explores outward from source s in concentric rings: distance 1 neighbors first, then distance 2, never jumping to ring d + 1 until every node in ring d has entered a First-In First-Out (FIFO) queue.

Mark Visited on Enqueue, Not Pop

A classic beginner bug is marking nodes visited only when popped from the queue. If two frontier vertices connect to node x before x is popped, x gets queued twice and bloats runtime. Always mark visited immediately upon pushing to the queue.

Because edges have equal unit weight (1 hop each), the very first wave reaching target t guarantees minimum hop count with total time O(V + E).

BFS INVARIANTS & APPLICATIONS

1. Data Structure: Strict FIFO Queue + visited boolean/parent map
2. Shortest Path: Unweighted single-source minimum edge distance in O(V + E)
3. Bipartite Check: Alternate 2-coloring across wave boundaries detects odd cycles
4. Level-Order Trees: Clean level grouping without recursion stack overflow

Try asking Zernexa AI "Explain why visited is marked on enqueue" to verify how quickly it cites this exact page.

3. Depth-First Search & Discovery Times

Following One Path to Its Limit

Depth-First Search (DFS) commits down outgoing edges recursively (or via an explicit LIFO stack) until reaching dead ends, then backtracks smoothly to explore remaining unvisited branches. DFS naturally produces connected components and full discovery/finish timestamps.

Three-Color State Tracking

Recording states as Unvisited (0), Visiting / On-Stack (1), and Completed (2) reveals directed graph edge classes immediately: meeting a node in Visiting (1) proves a genuine back-edge - and therefore a directed cycle.

Both recursive and iterative DFS run in O(V + E), though iterative stacks safeguard systems languages against deep call-stack limits on 100k-node chains.

EDGE CLASSIFICATION IN DIRECTED DFS

1. Tree Edge: Leads to an Unvisited (0) vertex - builds DFS forest
2. Back Edge: Points to an active ancestor (state 1) - proves a cycle exists
3. Forward Edge: Jumps ahead to an already finished descendant (state 2)
4. Cross Edge: Connects across distinct finished subtrees (state 2)

In undirected graphs, every non-tree edge is simply a back-edge to an ancestor (ignoring the immediate parent pointer).

4. Topological Ordering & Kahn’s Algorithm

Resolving Dependencies Cleanly

A Directed Acyclic Graph (DAG) admits a linear ordering of vertices such that every directed edge u -> v places task u strictly ahead of dependent task v. Build tools, spreadsheet formulas, and compiler schedules rely directly on topological orderings.

In-Degree Peeling (Kahn’s Algorithm)

Count incoming edges (inDegree[v]) for all vertices. Enqueue every vertex with inDegree === 0, pop them one by one into the sorted schedule, decrement neighbors’ inDegrees, and enqueue newly zeroed vertices. If fewer than |V| vertices finish, a dependency cycle blocked completion.

Alternative DFS post-order: reversing the order in which DFS completes vertices yields another valid O(V + E) topological order.

> **CYCLE & SCHEDULING FACTS**
>
> 1. A finite directed graph has a topological order IF AND ONLY IF it is a DAG
> 2. Topological orderings need not be unique - ties allow parallel execution
> 3. Longest path in a general graph is NP-hard, but O(V + E) DP in any DAG!
> 4. Kahn queue with min-heap produces the lexicographically smallest order

5. Weighted Shortest Paths (Dijkstra)

Greedy Relaxation with Min-Heap

When road edges carry non-negative travel times or weights w >= 0, BFS hop count can pick slow routes. Dijkstra maintains tentative best distances dist[v] in a binary min-priority queue, greedily locking the closest settled node u and relaxing edges dist[u] + w < dist[v].

Why Negative Weights Break Greedy Locking

Dijkstra assumes settling u guarantees no shorter route to u exists later. A negative edge downstream could retroactively undercut a settled vertex, requiring Bellman-Ford O(VE) relaxation rounds instead.

With binary heap priority queues and lazy deletion checks, Dijkstra finishes in O((V + E) log V) cleanly.

> **DIJKSTRA STEP-BY-STEP LOOP**
>
> 1. Initialize dist[start] = 0 and dist[others] = +Infinity; push (0, start)
> 2. Pop pair (d, u): if d > dist[u], skip stale duplicate heap entry immediately
> 3. For each edge (u, v, w): if dist[u] + w < dist[v], update dist[v] and push
> 4. Reconstruct optimal route by walking parent[t] backwards to source s

6. Disjoint Sets (DSU) & Spanning Trees

Near-Constant Connectivity Queries

Disjoint Set Union tracks partitioned clusters under two operations: find(x) and union(a, b). Pairing Path Compression (flattening parent pointers during lookup) with Union by Rank keeps amortized cost at O(alpha(N)) - effectively < 5 steps for any practical universe size.

Kruskal’s Minimum Spanning Tree

To wire N campuses with minimum total fiber length, sort all edges ascending by weight and iterate chronologically: if find(u) !== find(v), add the edge and merge components until N - 1 tree edges connect the whole graph.

Prim’s MST grows outward from one start node using a min-heap exactly like Dijkstra, excelling on denser graphs.

> **CONNECTIVITY ESSENTIALS**
>
> 1. Cycle detection in undirected incremental edges takes O(alpha(V)) per edge
> 2. Every spanning tree of V connected vertices contains exactly V - 1 edges
> 3. Cut Property: Minimum edge crossing any graph partition safely joins an MST
> 4. Kruskal total time is O(E log E) dominated strictly by sorting edges

7. All-Pairs Paths & Bellman-Ford

Bellman-Ford & Arbitrage Cycles

Relaxing every edge V - 1 times handles arbitrary negative weights in O(VE). Running one extra V-th relaxation pass detects negative-weight cycles instantly if any dist[v] decreases further - fundamental in currency arbitrage and routing stability checks.

Floyd-Warshall Matrix DP

When graphs are compact (|V| <= 400) and all-pairs shortest distances are queried repeatedly, iterating intermediate pivots k from 1..V computes dp[i][j] = min(dp[i][j], dp[i][k] + dp[k][j]) in clean triple nested loops.

A* search accelerates single-pair grid pathfinding by guiding priority order via f(n) = g(n) + h(n) admissible distance heuristics.

> **ALGORITHM SELECTION CHEAT SHEET**
>
> 1. Unweighted edges -> Standard BFS O(V + E)
> 2. Weighted >= 0 edges -> Dijkstra + Min-Heap O((V + E) log V)
> 3. Negative weights allowed -> Bellman-Ford O(VE)
> 4. All-pairs sparse small graph -> V x Dijkstra or Floyd-Warshall O(V^3)

8. Common Exam Traps & Checklist

Grid Matrices Are Implicit Graphs

Word ladders, island flooding, knight moves, and matrix shortest paths rarely build explicit adjacency arrays - compute 4-way or 8-way delta offsets dynamically, stay strictly inside [0..rows-1][0..cols-1], and guard visited flags carefully.

Multi-Source BFS Rotates Effortlessly

When questions ask for nearest exit distance from ANY gate simultaneously (like rotten oranges or nearest zero cells), seed ALL starting gates into the queue at distance 0 before popping a single element.

Before submitting any graph traversal solution, verify disconnected islands (outer loop over every vertex 0..N-1), self-loops, duplicate parallel edges, and single-node edge cases.

> **PRE-SUBMISSION EDGE CASE AUDIT**
>
> 1. Empty graph (N = 0, E = 0) and single-vertex disconnected island checks
> 2. Directed vs. undirected symmetry when adding adjacency entries
> 3. Visited marked at enqueue time in BFS; recursion depth vs iterative stack in DFS
> 4. BigInt / 64-bit accumulator check for large summed path costs